

Databento Data Quality Report — LE (Live Cattle), 2026-07-15

First completed asset from the Databento historical backfill (WORKFLOW.md Phase 4). Explored before writing the transform/join stage, specifically to catch data-shape surprises before they got baked into pipeline code. Two real findings changed the transform-stage design (logged in WORKFLOW.md Phase 3/4); this report is the detailed evidence behind those changes, plus everything else checked along the way.

Source files: Data/databento_raw/CME_Globex_MDP3.0_LE_FUT_Definition.zip, CME_Globex_MDP3.0_LE_FUT_OHLCV.zip — GLBX.MDP3 dataset, LE.FUT parent symbol, `stype_in="parent"`, 2010-06-06 → 2026-07-13. One CSV (Zstandard-compressed) per calendar day per schema: 4,050 ohlcv-1d files, 5,036 definition files.

Tooling: [polars](#) (not pandas — see “Why polars” at the end).

Headline findings

1. **The raw symbol field is decade-ambiguous — do not use it as a contract key.** CME’s raw symbology uses a single-digit year code, which collides across a pull this long. Confirmed and fixed (see below); this was the most consequential finding.
2. **A parent-symbol query returns mostly spread/butterfly instruments, not outright** — 79% of rows for this asset. Not a targeting mistake; genuinely how CME organizes the instrument universe. The transform-stage design now keeps all three instrument classes instead of discarding 79% of the pull (see WORKFLOW.md Phase 3/4).
3. **Otherwise clean.** Zero non-positive prices, zero zero-volume outright rows, zero OHLC-relationship violations (`high >= low/open/close`, `low <= open/close`), 0.39% session coverage gap overall — and even that residual gap is mostly explained by calendar-library staleness (Juneteenth) rather than real missing data. One genuine 5-day gap found and isolated (below).

1. Symbol decade-ambiguity (critical)

CME’s raw symbol format is {ROOT}-{MONTH_CODE}-{SINGLE_DIGIT_YEAR} — e.g. LEZ5 for a December contract in a year ending in 5. Over a 16-year pull (2010–2026), that digit is ambiguous: both December 2015 *and* December 2025 report as LEZ5.

Confirmed directly on the highest-volume outright observed:

instrument_id	Date range	Close range	Real contract
7780	2014-07-02 → 2015-12-31	\$118.55 – \$159.88	December 2015
42041124	2024-07-02 → 2025-12-31	\$173.35 – \$247.75	December 2025

Both report `symbol == "LEZ5"`. The price ranges alone confirm these are different contracts (2015 vs. the 2024-2025 record-high cattle market), not a data artifact. **47 of 60** raw outright symbols observed had exactly 2 distinct `instrument_ids` for this reason — every contract-month whose 10-years-earlier sibling also falls inside the pull window.

Fix, verified clean: the `definition` schema's `maturity_year` field is an unambiguous 4-digit integer (confirmed 2015 vs. 2025 for the two `instrument_ids` above). Joining `ohlcv-1d → definition` on `(date, instrument_id)` and building `contract_symbol = {asset}{month_code}{maturity_year}` instead of trusting the raw `symbol` string resolves it completely:

- Raw unique symbol strings: **60**
- Unique disambiguated `contract_symbols`: **107** ($= 47 \times 2 + 13 \times 1$ — exact match with the collision count above)
- `contract_symbols` with more than one `instrument_id` after the fix: **0**

`instrument_id` itself is *not* the problem — it never gets reused across contract instances (confirmed: 7780 only ever appears for the 2015 contract's trading life, 42041124 only for 2025's). It's specifically the symbol string's year digit that's lossy. Any future use of this data — outright, spread, or butterfly — needs the same `maturity_year`-based fix; the collision applies equally to spread/butterfly symbols (LEV6-LEZ6, LE:BF Z6-G7-J7) since they're built from the same raw month codes.

2. Parent-symbol queries bundle spreads and butterflies

`instrument_class` (from `definition`, authoritative — not a regex guess on the symbol string) splits the joined data as:

Instrument class	Rows	% of total
Outright futures (F)	35,142	21.1%
Spread/combo (S) — calendar spreads + butterflies	131,122	78.9%

Splitting **S** further by leg count (butterflies carry a **:BF** marker in their symbol):

Type	Rows	Unique symbols	Example
Outright	35,142	107 (disambiguated)	LE Dec 2026
Calendar spread (2-leg)	108,996	807	LEV6-LEZ6
Butterfly (3-leg)	22,126	84	LE:BF Z6-G7-J7

This happens because `stype_in="parent"` asks Databento for everything CME lists under the **LE** product — and CME lists exchange-tradeable spread and butterfly instruments directly, not just outright, so traders don't have to leg in manually. Pairs combinatorially outnumber individual months, and triplets outnumber pairs, which is why spreads+butterflies dwarf outright by rows even though there are fewer *distinct* butterfly listings than spread listings per month-set.

This is not wasted data. The calendar spread prices are a direct, exchange-traded quote on $F_{\text{near}} - F_{\text{deferred}}$ — the exact quantity the carry formula (`WORKFLOW.md` Phase 4) needs — without the leg-alignment risk of back-differencing two separately-sourced outright prices. Sample (LEV6-LEZ6, 2015):

file_date	symbol	close	volume
20150702	LEV6-LEZ6	-0.500	11
20150708	LEV6-LEZ6	-0.850	2
20150710	LEV6-LEZ6	-0.350	7

Small-magnitude, real-looking spread quotes (order of \$0.30–0.85/cwt) with genuine trade volume — usable as-is once the same disambiguation fix is applied to the two symbol legs. Butterflies are a distinct, currently out-of-scope signal (curve curvature) — logged as a candidate in `WORKFLOW.md` Phase 3, not built.

Note for future asset pulls: Databento almost certainly bills by data volume, and 79% of this pull was unusable for the outright term structure specifically (now usable for carry, per above, but still worth knowing before repeating this exact query shape across the other 32 CME assets).

3. Definition schema has duplicate rows per (date, instrument_id)

The join fans out slightly: 166,264 joined rows vs. 166,093 source `ohlcv-1d` rows (+171, or +0.10%). Root cause: **1,648** (date, instrument_id) pairs in the definition files

have more than one row — genuinely identical duplicates, not conflicting values. Example (`instrument_id=42057126`, 2025-11-16): 3 byte-for-byte identical rows (`raw_symbol`, `instrument_class`, `maturity_year` all match).

This is consistent with `definition` being an event-sourced schema (each row is a security-definition *message*, not a deduplicated daily snapshot) — CME appears to occasionally republish an unchanged definition intraday. Harmless for our purposes (the duplicates carry identical values, so the join isn’t corrupted, just slightly inflated), but the transform script should `.unique()` the definition frame on (`date`, `instrument_id`) before joining, rather than silently accept the row inflation. Self-consistency check: the 36-row difference between this session’s polars-with-join outright count (35,142) and an earlier pandas-with-regex-only count (35,106, no join) matches this artifact almost exactly.

4. OHLC / price sanity — clean

On the 35,142 outright rows:

- Non-positive `open/high/low/close`: **0**
- Zero-volume rows: **0** (every outright row has real trade volume)
- Rows violating `high >= low/open/close` or `low <= open/close`: **0**
- Duplicate (`date`, `contract_symbol`) rows (post-disambiguation): **36** — fully explained by the definition-duplication artifact in section 3, not a real duplicate print.
- Price scale confirmed: raw integers are fixed-point, $\times 1e-9$ (Databento’s standard DBN price scale) — scaled outright closes range \$76.60–\$258.48, consistent with real Live Cattle prices across 2010-2026.

5. Date coverage

Checked against the real exchange calendar (`pandas_market_calendars`, `CMEGlobex_Livestock` — the same calendar the QA dashboard’s OHLCV Coverage page uses for the yfinance-sourced panel):

	Count
Expected trading sessions, 2010-06-07 → 2026-07-13	4,058
Actual distinct days with outright data	4,050
Missing (calendar expected, no data)	16 (0.39%)
“Extra” (data present, calendar didn’t expect a session)	8

Count

Of the 16 missing:

- **5 are Juneteenth** (2022-06-20, 2023-06-19, 2024-06-19, 2025-06-19, 2026-06-19). CME doesn't trade Live Cattle on Juneteenth (a federal holiday since 2021); the CMEglobex_Livestock calendar package hasn't been updated to reflect it. **This is a gap in our own calendar tooling, not in the Databento data** — a real, concrete instance of exactly the caveat already written on the dashboard's OHLCV Coverage page ("a calendar mismatch... worth spot-checking outliers rather than treating every missing session as a confirmed defect").
- **6 are explained by condition.json** reporting degraded/missing status for those dates (Databento's own data-quality flag).
- **5 are a genuine, unexplained gap: 2012-02-06 through 2012-02-10** (a full trading week). Verified directly — both ohlcv-1d *and* definition files are simply absent for all 5 dates (not empty; the files don't exist), while 2012-02-03 and 2012-02-13 immediately bracket it with normal data. condition.json claims available for these dates, so Databento's own metadata doesn't flag this as a known gap. **Worth raising with Databento if 2012-era history matters for anything specific** — flagged here rather than silently accepted, per this project's own standing rule about labeling gaps honestly instead of patching around them.

The 8 “extra” dates (data present on days the calendar didn't expect) are consistent with actual abbreviated holiday sessions CME held that the generic calendar package marks as fully closed (e.g. 2010-07-05, 2010-11-25 Thanksgiving, 2011-01-17 MLK Day) — another instance of the calendar package being the imprecise side, not the data.

6. Why polars (not pandas) for this exploration

Measured, this session, reading + joining all 9,086 files (~948k total rows across both schemas):

Step	Time
Read 4,050 OHLCV files (166,093 rows)	2.5s
Read 5,036 definition files, 8 of 65 columns selected (782,482 rows)	29.5s
Join on (date, instrument_id)	0.97s

Step	Time
Total, including all checks in this report	35.1s

This was a single-asset, ~200MB exploration — not a scale where pandas would have struggled. The reason to do it in polars anyway: this exploration is the prototype for the real transform script, which has to run this same join+filter logic across the **full 33-CME-asset backfill** (roughly 30-40x this volume — likely tens of millions of definition rows alone). Polars’ multi-threaded CSV parsing and column-selection-on-read (only 8 of 65 definition columns were ever materialized) are what pay off at that scale, not at this one. Scoped to `databento/` only — the rest of the repo stays pandas (`cleanup.md`).

Recommendations for the transform script (`databento/`, not yet built)

1. Join `ohlcv-1d` → `definition` on `(date, instrument_id)`, not a static `instrument_id` → `symbol` map — `definition` must be deduplicated on that same key first (section 3).
2. Derive `contract_symbol` from `maturity_year/maturity_month`, never from the raw `symbol` string (section 1) — applies to outrights, spreads, and butterflies alike.
3. Write three outputs, not one, all keyed by the same disambiguated contract identity (`WORKFLOW.md` Phase 3/4):
 - `term_structure.parquet` — outrights
 - `term_structure_spreads.parquet` — calendar spreads
 - `term_structure_butterflies.parquet` — butterflies
4. Don’t assume `condition.json` catches every gap — the Feb-2012 gap (section 5) shows Databento’s own metadata can miss a real hole. A basic post-transform coverage check against a real exchange calendar (the same pattern the QA dashboard already uses) is worth running per asset, not just for LE.
5. Prefer calendar-spread prices over back-differenced outrights for carry construction once `term_structure_spreads.parquet` exists (section 2).

Appendix A: OHLCV-1D schema, field by field

10 columns. One row per (instrument, trading day).

Column	What it is	Example (LEZ6, 2026-07-13)
<code>ts_event</code>	Nanosecond timestamp (since 1970-01-01 UTC) marking the trading session this bar covers.	1783900800000000000 → 2026-07-11
<code>rtype</code>	Databento’s internal record-type discriminator — identifies which schema this row belongs to. Constant within a single <code>ohlcv-1d</code> pull.	35
<code>publisher_id</code>	Numeric ID for the specific dataset+venue combination (Databento’s publisher registry).	1 (= GLBX.MDP3, CME Globex)
<code>instrument_id</code>	Databento/CME’s numeric ID for this specific contract <i>instance</i> . Confirmed stable for a contract’s entire life, never reused for a different contract (Section 1) — but not usable as a cross-time symbol lookup without the <code>definition</code> join.	42005708
<code>open</code>	Session opening price. Fixed-point integer, $\div 1e9$ = real price.	230550000000 → 230.55
<code>high</code>	Session high. Same scaling.	232550000000 → 232.55
<code>low</code>	Session low. Same scaling.	230375000000 → 230.375
<code>close</code>	Session close/settlement price. Same scaling.	230825000000 → 230.825
<code>volume</code>	Contracts traded that session.	5569
<code>symbol</code>	Plain integer, no scaling. The resolved, human-readable ticker for this instrument on this date. Present only because the job was submitted with <code>map_symbols: true</code> — decade-ambiguous, see Section 1.	LEZ6

Appendix B: Definition schema, field by field

65 columns. One row per instrument per publication event — not deduplicated per day (Section 3). Values below are from a real row: the December 2026 Live Cattle outright (`raw_symbol=LEZ6`, `instrument_id=42005708`), as published 2026-07-13.

A note on the huge sentinel numbers below (2147483647, 9223372036854775807, 65535, 255, 127): these are not malformed data. Databento’s underlying binary format (DBN) uses fixed-width integers, and its convention for “not applicable to this instrument” is the maximum value representable in that field’s width, carried through literally into the CSV rather than left blank:

Sentinel	Bit width	Meaning
9223372036854775807	signed 64-bit max ($2^{63}-1$)	Field not applicable (e.g. <code>strike_price</code> on a future)
2147483647	signed 32-bit max ($2^{31}-1$)	Field not applicable (e.g. <code>contract_multiplier</code> on this product)
65535	unsigned 16-bit max ($2^{16}-1$)	Field not applicable (e.g. <code>decay_start_date</code>)
255	unsigned 8-bit max (2^8-1)	Field not applicable (e.g. <code>maturity_day</code> when only year+month matter)
127	signed 8-bit max (2^7-1)	Field not applicable (e.g. <code>contract_multiplier_unit</code>)

Confirmed by example: `strike_price`, `price_ratio` = 9223372036854775807 — correctly null, since outright futures don’t have a strike price (that field exists in this schema because `definition` covers CME options too, in the same table shape).

1. Identity & versioning — which record is this, has it changed

Column	What it is	Example
<code>ts_recv</code>	Nanosecond timestamp Databento received this record.	1783900800000000000
<code>ts_event</code>	Nanosecond timestamp CME generated this record.	1783857868281000000
<code>rtype</code>	Record-type discriminator for the <code>definition</code> schema.	19
<code>publisher_id</code>	Venue/feed ID.	1 (GLBX.MDP3)

Column	What it is	Example
<code>instrument_id</code>	CME's numeric ID for this instrument instance. Stable for life; the join key used throughout this report.	42005708
<code>raw_symbol</code>	CME's raw ticker string — the decade-ambiguous one (Section 1).	LEZ6
<code>symbol</code>	Same as <code>raw_symbol</code> in this pull (both decade-ambiguous).	LEZ6
<code>security_update_action</code>	Single-letter code for what kind of update this record is (A=add, M=modify, others exist for delete). Explains why duplicate rows exist per day (Section 3) — each republish is its own “update” event.	A

2. What kind of instrument this is

Column	What it is	Example
<code>instrument_class</code>	The field the transform filters on: F=outright future, S=spread/combo (both calendar spreads and butterflies, Section 2).	F
<code>security_type</code>	Plain-text instrument type.	FUT
<code>cfi</code>	ISO 10962 Classification of Financial Instruments code — a standardized 6-character instrument taxonomy (first letter F=Futures; the remaining letters encode finer detail this report doesn't decode with full confidence).	FCAXSX
<code>group</code>	Product group code — matches the asset root for this product.	LE

Column	What it is	Example
exchange	ISO 10383 Market Identifier Code (MIC).	XCME
asset	Product root — what this project keys <code>contract_symbol</code> on.	LE
underlying	The underlying instrument, for derivatives-of-an-instrument (e.g. options on a future). Null for an outright future.	<i>(empty)</i>
securitiesubtype	Further security subtype classification. Null here.	<i>(empty)</i>
underlying_product	Numeric code for the underlying product family.	2

3. Contract economics — how to interpret a raw price or quantity

Column	What it is	Example
contract_multiplier	Dollar value per price point, for products that use this convention. Null (sentinel) for Live Cattle — this product expresses size via <code>unit_of_measure_qty</code> instead.	2147483647 (null)
contract_multiplier_unit	Unit the multiplier above is expressed in. Null here.	127 (null)
unit_of_measure	The physical unit prices are quoted against.	LBS
unit_of_measure_qty	Contract size in that unit. Fixed-point, $\div 1e9$. Verified: 40,000 lbs — the real, correct CME Live Cattle contract size.	40000000000000 $\rightarrow$ 40,000
price_ratio	A price-conversion ratio used for certain product types. Null (sentinel) here.	9223372036854775807 (null)

Column	What it is	Example
<code>display_factor</code>	A <i>separate</i> scale factor from the universal $\div 1e9$ used for OHLCV/limit prices — applies to currency- <i>amount</i> -typed fields like <code>min_price_increment_amount</code> below. Confirmed by arithmetic, not assumed (see the tick-value check below).	1000000
<code>min_price_increment</code>	The tick size, in quoted price units. Fixed-point, $\div 1e9$.	25000000 $\rightarrow$ 0.025
<code>min_price_increment_amount</code>	Dollar value of one tick. Scaled by <code>display_factor</code> , not the standard $\div 1e9$. Verified two independent ways: (a) 0.025 (tick, quote units) $\times$ 40,000 lbs $\div$ 100 = \$10.00; (b) 10,000,000 $\div$ <code>display_factor</code> (1,000,000) = \$10.00. Both match, and both match the real, documented CME Live Cattle tick value.	10000000 $\rightarrow$ \$10.00

4. Lifecycle dates — when this specific contract exists

Column	What it is	Example
<code>activation</code>	Nanosecond timestamp this contract was first listed. Decoded: 2025-06-06 21:30 UTC — about 18 months before expiration, consistent with CME's typical multi-year-out listing cycle for Live Cattle.	1749245400000000000

Column	What it is	Example
expiration	Nanosecond timestamp this contract stops trading. Decoded: 2026-12-31 18:00 UTC.	1798740000000000000
maturity_year	Unambiguous 4-digit contract year — the field that fixes the decade-ambiguity problem (Section 1).	2026
maturity_month	Contract month, 1-12.	12
maturity_day	Contract day, for products with day-specific maturities. Null (sentinel) for a standard monthly future.	255 (null)
maturity_week	Contract week, for weekly-maturity products. Null here.	255 (null)
trading_reference_date	The date trading_reference_price below was struck. Encoded differently from the timestamp fields above — as a day count since 1970-01-01 , not nanoseconds. Decoded: 2026-07-10 (the prior trading day relative to this file's 2026-07-13 date).	20644
decay_start_date	Start date for instruments with an amortizing/decaying notional (e.g. some fixed-income products). Null (sentinel) — not applicable to a physically-settled livestock future.	65535 (null)

5. Trading limits & risk parameters — exchange-set guardrails, not market prices

Column	What it is	Example
<code>high_limit_price</code>	Upper bound of the day's allowed price band. Fixed-point, $\div 1e9$.	238775000000 $\rightarrow$ 238.775
<code>low_limit_price</code>	Lower bound of the day's allowed price band.	221775000000 $\rightarrow$ 221.775
<code>max_price_variation</code>	Exchange parameter used to compute the limit band above/below the reference price. This report did not fully reverse-engineer the exact formula relating this value to the limit prices above — flagged rather than guessed.	975000000
<code>trading_reference_price</code>	The settlement/reference price the limit band is centered on.	230275000000 $\rightarrow$ 230.275

6. Lot sizing — minimum/maximum tradeable quantities

Column	What it is	Example
<code>min_lot_size</code>	Minimum order size. 0 observed — likely means “no restriction beyond the exchange default” rather than a literal zero-size order; not fully confirmed.	0
<code>min_lot_size_block</code>	Minimum size for a privately-negotiated block trade.	0
<code>min_lot_size_round_lot</code>	Minimum size considered a standard “round lot.”	0
<code>min_trade_vol</code>	Minimum tradeable volume on the open market.	1
<code>max_trade_vol</code>	Maximum single-order size.	500

7. Currency

Column	What it is	Example
<code>currency</code>	Quote currency.	USD
<code>settl_currency</code>	Settlement currency, when different from quote currency. Null (same as <code>currency</code>) here.	<i>(empty)</i>

8. Options-specific fields — present in the schema, not applicable to futures

Column	What it is	Example
<code>strike_price</code>	Option strike price. Null (sentinel) — definition is a shared schema across CME futures <i>and</i> options; these fields exist for the option rows, not this one.	9223372036854775807 (null)
<code>strike_price_currency</code>	Currency the strike is quoted in. Null here.	<i>(empty)</i>

9. Market microstructure & order-matching mechanics

Column	What it is	Example
<code>match_algorithm</code>	Single-letter code for which order-matching algorithm this instrument uses (CME publishes several — FIFO, pro-rata, and others). This report did not confirm the exact code-to-algorithm mapping with full certainty — flagged rather than guessed.	K
<code>md_security_trading_status</code>	Current trading status code (halted, open, etc.), as of this definition snapshot. Null (sentinel) — status is carried in a separate real-time status message, not this reference record.	255 (null)

Column	What it is	Example
<code>market_depth</code>	Number of price levels disseminated in the outright order book.	5
<code>market_depth_implied</code>	Number of price levels disseminated for implied (derived-from-spread) pricing.	2
<code>market_segment_id</code>	Numeric ID for the CME market segment this instrument trades in.	70
<code>channel_id</code>	Numeric ID for the specific data channel this instrument is disseminated on.	6

10. Legacy / lower-relevance fields for this project

Real CME/Databento fields, not currently needed by this project's transform:

Column	What it is	Example
<code>inst_attrib_value</code>	A bitmask of instrument attribute flags (e.g. electronic-eligible, implied-pricing-eligible). This report did not decode individual bits.	794699
<code>underlying_id</code>	<code>instrument_id</code> of the underlying instrument, for derivatives-of-an-instrument. 0 for an outright.	0
<code>raw_instrument_id</code>	CME's own raw instrument ID, as opposed to Databento's <code>instrument_id</code> . Identical value in this row.	42005708
<code>appl_id</code>	Internal CME application/system ID.	316
<code>decay_quantity</code>	Notional decay amount, for amortizing instruments. Null (sentinel).	2147483647 (null)

Column	What it is	Example
<code>original_contract_size</code>	Original size before any decay, for amortizing instruments. Null (sentinel).	2147483647 (null)
<code>main_fraction</code>	Numerator for fractional price display (e.g. bond prices quoted in 32nds). Null (sentinel) — Live Cattle is decimal-quoted.	255 (null)
<code>sub_fraction</code>	Denominator refinement for fractional price display. Null (sentinel).	255 (null)
<code>price_display_format</code>	Code for how price should be displayed (decimal vs. fractional). Null (sentinel).	255 (null)
<code>settl_price_type</code>	Code classifying the type of settlement price (e.g. final vs. preliminary). A real, populated code here — not decoded further in this report.	3
<code>flow_schedule_type</code>	Code relating to physical delivery/flow scheduling, relevant to some physically-delivered products. Null (sentinel) here.	127 (null)
<code>tick_rule</code>	Code for any non-uniform tick-size rule (e.g. different tick sizes at different price levels). Null (sentinel) — Live Cattle has a single uniform tick.	255 (null)
<code>user_defined_instrument</code>	Flag (Y/N) for whether this is an exchange-listed instrument or a user-defined combination. N — this is a real, exchange-listed instrument.	N